Chapter 9

CS1 course package - Montclair State University (MSU)

Jiayin Wang[1] and Michelle Zhu [2]

[1]Montclair State University
[2]Kennesaw State University

PDC Course Material Used in CS1 at Montclair State University

Abstract

This course exemplar describes the integration of an unplugged data-parallelism activity in CSIT 111, Fundamentals of Java Programming, a CS1-level Java programming course at Montclair State University. The activity uses a paper-based Flag Maker task, supported by short browser-based animations, to introduce students to foundational parallel and distributed computing (PDC) concepts without requiring specialized hardware, parallel programming libraries, or a separate programming assignment. Students physically model processors that color a grid representation of the flag of Mauritius under different task-partitioning strategies. By comparing a single-processor execution, a two-processor execution, a four-processor stripe-based execution, and a four-processor column-based execution, students experience sequential execution, parallel execution, speedup, synchronization, load imbalance, race conditions, and pipelining in a concrete and accessible way. The activity is embedded within the existing CSIT 111 course structure and is accompanied by a pre-activity quiz, post-activity quiz, pre-course survey at the beginning of the semester, and post-course survey at the end of the course. Results from paired pre/post survey responses show positive gains in students' self-reported understanding of parallel processing, distributed computing, and PDC-related confidence, suggesting that even a short unplugged intervention can provide meaningful early exposure to PDC concepts in CS1.

Descriptor Elements

Programming Language Employed: Java

Textbook Used: Revel for Java Software Solutions, Foundations of Program Design, 10th Edition by Lewis & Loftus[1]

Relevant core courses: CS1: CSIT 111 Fundamentals of Java Programming

Pre-requisites: CS0: CSIT 104 Python Programming I

Relevant PDC topics: Table 9.1 lists the integrated PDC topics with bloom's classification (Remember (RE), Understand (UN), Apply (AP), Analyze (AN), Evaluate (EV), and Create (CR)). Because CS1 uses Flag Maker activity as an unplugged conceptual exercise, the Bloom levels are limited to Remember (RE) and Understand (UN).

Student Learning outcomes (SLO):

Table 9.1: Relevant PDC topics and Bloom's levels

PDC Concept	Chapter Section 9.3.1
Data parallelism	UN
Sequential vs. parallel execution	UN
Speedup	RE
Scalability	RE
Load balancing and idle time	UN
Synchronization	RE
Contention for shared resources	RE
Race conditions	RE
Pipelining	RE
Distributed task partitioning	UN

1. Students will be able to distinguish sequential execution from parallel execution.
2. Students will be able to explain how dividing work among multiple processors can affect completion time, speedup, scalability, and load balance.
3. Students will be able to recognize when an algorithmic solution can benefit from the use of multiple threads that communicate.
4. Students will be able to identify common PDC issues illustrated by the activity, including synchronization, race conditions, contention, dependencies, idle time, and pipelining.

Context for use: Montclair State University, located 12 miles west of midtown Manhattan, is New Jersey's largest Hispanic-Serving Institution (HSI) and a public R2 doctoral research university enrolling approximately 23,000 students. The Flag Maker activity was implemented in CSIT 111, Fundamentals of Java Programming. CSIT 111 is a CS1-level introductory Java programming course offered by the School of Computing and required for both the BS in Computer Science and the BS in Information Technology. Table 9.2, Table 9.3, and Table 9.4 summarize the demographic characteristics of students who completed the pre-course survey in Spring 2025 and Fall 2025. Specifically, the tables report the distribution of students by gender, race/ethnicity, and current class standing, providing context for interpreting the survey results across the two semesters.

The course is taught over a 14-week semester, with approximately seven sections offered each semester and an average class size of 32 students. The course introduces fundamental programming concepts, including primitive data types, expressions, conditionals, loops, arrays, classes, objects, and object-oriented program design. The revised version of CSIT 111 retains the standard CS1 structure while adding explicit exposure to data parallelism. The updated course overview states that students are introduced to the fundamentals of data parallelism and to how programs can efficiently use modern multi-core processors. The revised learning outcomes also include recognizing when an algorithmic solution can benefit from the use of multiple communicating

threads. The Flag Maker activity was placed in Class 23 out of 24 classes as a data parallelism activity. By this point in the course, students had already encountered core CS1 ideas. In this implementation, the activity is used as an unplugged in-class exercise supported by browser-based animations. Students use printed grids, coloring materials, and timing data to model processors working on the same task under different partitioning strategies. The activity is designed for instructors who want a low-barrier way to introduce PDC concepts in CS1 without displacing major Java programming content.

Table 9.2: MSU CS1 Gender Distribution

Gender	SP25 ($n = 13$)	FA25 ($n = 34$)
Female	3	8
Male	10	25
Not reported	0	1
Total	13	34

Table 9.3: MSU CS1 Race/ethnicity Distribution

Race/Ethnicity	SP25 ($n = 13$)	FA25 ($n = 34$)
Asian / Pacific Islander	0	1
Black/African American	4	12
Hispanic	4	9
Middle Eastern or North African	0	3
White	3	5
Declined to specify	1	3
Not reported	1	1
Total	13	34

Table 9.4: MSU CS1 Current Class Standing Distribution

Current Class Standing	SP25 ($n = 13$)	FA25 ($n = 34$)
First year (freshman)	7	12
Sophomore	2	13
Junior	2	5
Senior	2	4
Total	13	34

9.1 Introduction

Parallel and distributed computing (PDC) is now central to modern computing, yet many CS1 courses still introduce programming primarily through sequential execution. Students learn how to write programs that proceed step by step, but they often have limited opportunities to reason about how work can be divided, executed concurrently, and coordinated across multiple processing units. Introducing these ideas early can help students develop a more accurate mental model of contemporary computing systems.

At the same time, CS1 courses are already full of essential topics, including variables, expressions, conditionals, loops, arrays, classes, objects, and problem-solving. Therefore, adding PDC content should not require removing core programming material or introducing advanced parallel programming tools too early. The goal of this activity is to provide a lightweight, accessible entry point into PDC through an unplugged classroom exercise that complements, rather than competes with, the existing Java curriculum.

The Flag Maker activity introduces data parallelism by asking students to physically model processors coloring a grid representation of the flag of Mauritius. By comparing single-processor and multi-processor versions of the same task, students observe how parallel execution can reduce completion time while also creating new challenges, such as synchronization, contention, race conditions, and coordination overhead. Browser-based animations are used to reinforce the connection between the physical activity and the underlying computing concepts.

This section presents the design, implementation, and assessment of the Flag Maker activity in CS1. The activity is assessed through pre- and post-activity quizzes, while broader course-level changes are examined through beginning-of-semester and end-of-course surveys.

9.2 How CS1 was changed and PDC topics integrated

The integration strategy preserved the original CS1 structure while adding a focused PDC learning experience. The pre-integration syllabus presented CS1 as a Java programming course covering digital computers, Java syntax and semantics, algorithms, logic, design, testing, documentation, and ethics in computing. Its course outcomes focused on program creation, assignments and expressions, Java library use, classes and objects, structured programming, arrays, objects, conditionals, and loops. The revised syllabus retained these core CS1 goals but added explicit PDC language in the course overview and learning outcomes. In particular, the revised course overview states that the course introduces the fundamentals of data parallelism and prepares students to understand how programs can efficiently use modern multi-core processors. The revised student learning outcomes also add an outcome requiring students to recognize when an algorithmic solution can benefit from multiple threads that communicate.

9.2.1 Integrated Syllabi (Pre and Post)

The pre- and post-integration syllabi for CS1 show that the course remained a CS1 Java programming course. The main course description and programming goals were not

substantially changed. Both versions emphasize Java syntax and semantics, programming logic, algorithmic problem solving, conditionals, loops, arrays, classes, objects, and object-oriented program design. The purpose of the integration was therefore not to redesign the entire course, but to introduce PDC concepts within the existing CS1 structure.

The main differences between the pre- and post-integration syllabi were structural. The earlier version of CS1 was organized as a 16-week course with 75-minute class meetings, while the revised version was organized as a 14-week course with 85-minute class meetings. In addition, the in-class lab assignments were combined and reduced from 10 labs to 6 labs. This revised structure created space for a lightweight PDC intervention without replacing the core Java programming sequence. The key new integration point is Module 9, Parallel Computing, where the Flag Maker data parallelism activity was implemented as an in-class unplugged activity supported by browser-based animations and pre/post activity assessment.

Table 9.5: Summary comparison of CS1 before and after PDC integration.

Syllabus Element	**Pre-Integration Version**	**Post-Integration Version**
Primary language	Java	Java
Semester format	16 weeks, 75 minutes per class	14 weeks, 85 minutes per class
Core CS1 content	Java syntax, algorithms, logic, conditionals, loops, arrays, classes, objects, and object-oriented design	Java syntax, algorithms, logic, conditionals, loops, arrays, classes, objects, object-oriented design, **inheritance**, **PDC**
Lab structure	10 in-class lab assignments	6 combined in-class lab assignments
PDC content	Not explicitly included as a course module	**Module 9: Parallel Computing**
PDC activity	Not applicable	Flag Maker data parallelism activity with animations
PDC assessment	Not applicable	Pre-activity quiz, post-activity quiz, and course-level pre/post surveys

Table 9.6 summarizes the post-integration course schedule. The PDC component is intentionally limited in scope and placed near the end of the semester, after students have encountered the core programming constructs needed to reason about sequential execution, loops, conditionals, arrays, objects, and problem decomposition. The highlighted row shows the main syllabus-level integration of PDC content.

Overall, the syllabus integration was deliberately modest. The post-integration version preserved the traditional CS1 programming sequence while adding a short PDC module and an unplugged Flag Maker activity. This design allowed students to encounter data parallelism and related PDC concepts without requiring a separate parallel programming unit or reducing coverage of core Java topics.

9.2.2 Highlights

The CS1 integration is intentionally lightweight. It does not require parallel hardware, specialized software, or a new programming environment. Students use paper grids and coloring materials to physically model how a computer might execute a task under different processor configurations. The instructor then uses short browser-based animations to reinforce the mapping between the physical activity and the computing concepts.

Table 9.6: Post-integration CS1 course schedule with PDC integration highlighted.

Module	Week	Activities	Assignments
Module 1: Introduction	Week 1	Slides: Class 1–2	Homework 1
Module 2: Data and Expression	Weeks 2–3	Slides: Class 3–4	Lab 1; Homework 2
Module 3: Using and Writing Classes and Objects	Weeks 3–5	Slides: Class 6–8	Lab 2; Homework 3 and 4
Module 4: Conditional and Loops	Weeks 6–7	Slides: Class 10–12, 14	Lab 3; Homework 5; Lab 4; Homework 6
Module 5: Midterm Exam	Week 8	Slides: Class 16; Midterm Review; Midterm Exam	None
Module 6: Object-Oriented Design	Week 9	Slides: Class 17–18	Lab 5; Homework 7
Module 7: Array	Week 10	Slides: Class 20–21	Lab 6; Homework 8
Module 8: Inheritance	Weeks 11–12	Slides: Class 22	Homework 9
Module 9: Parallel Computing	**Week 12**	**Slides: Class 23; Flag Maker data parallelism activity; animations**	**Pre-activity quiz and post-activity quiz**
Module 10: Final Exam	Weeks 13–14	Slides: Class 24; Final Review; Final Exam Part 1 and 2	None

The activity was designed to fit within a normal class period. Students first complete a pre-activity quiz. The instructor then introduces the flag-coloring rules and facilitates several timed scenarios. After each scenario, students discuss what happened, why performance changed, and what new problems appeared as more processors were introduced. The activity concludes with animation-based reinforcement and a post-activity quiz.

9.2.3 Challenges

The main challenge was classroom logistics. The instructor had to prepare enough printed grids, markers or crayons, timing devices, and grouping instructions. Because each scenario used a different number of students per group, the instructor needed to decide group sizes before class and prepare extra copies for uneven enrollment. It was also important to explain that the pre-activity quiz was diagnostic rather than punitive, since students might not yet know the PDC terms being assessed. Finally, the instructor needed to reserve time for discussion after each scenario. Without structured reflection, students might have experienced the activity as only a coloring exercise rather than as a model of parallel computation.

Another challenge was the lengthy IRB approval process. Project activities were delayed while approval was pending, and because the approved protocol did not permit us to offer bonus points as an incentive, student participation in the research activities was relatively low. Administrative support was also needed to propagate the intervention consistently across all sections. The course was taught by a mix of full-time faculty and part-time lecturers, which made securing buy-in from every instructor and maintaining consistent content delivery an ongoing challenge.

9.3 New Unplugged Activities

9.3.1 Flag Maker

Problem Description, Prerequisites, and Learning Outcomes

In CS1, the activity of Flag Maker was adopted as an unplugged in-class data parallelism exercise supported by browser-based animations. Students acted as processors and colored a grid representation of the flag of Mauritius under different task-partitioning strategies, including single-processor, two-processor, four-processor stripe-based, and four-processor column-based execution.

The activity required no prior knowledge of parallel programming. Students only needed basic CS1-level understanding of sequential execution, loops, conditionals, and task decomposition. This made the activity appropriate for an introductory programming course because students could reason about parallel execution before learning formal multithreading APIs.

Based on the classroom implementation, the main learning goal was conceptual understanding rather than programming implementation. Students were expected to distinguish sequential and parallel execution, explain how data can be partitioned across processors, compare completion time across different numbers of processors, and recognize common PDC issues such as load imbalance, contention, and pipelining. The activity also helped students connect familiar CS1 ideas, such as loops and row/column-based data organization, to the way modern computing systems divide and coordinate work.

For more information, please refer to Chapter 1, Section 1.3.1 for the Flag Maker activity and Chapter 1, Section **??** for the browser-based animation of Flag Maker.

Implementation Details

The Flag Maker activity was implemented as a guided in-class unplugged exercise. The instructor used lecture slides to lead students through the activity sequence and to introduce the related PDC concepts after each scenario. The activity began with a pre-activity quiz, which was administered before any PDC concepts were introduced. After the activity, students completed a post-activity quiz with the same questions, followed by a short feedback survey about the Flag Maker activity.

During the activity, students were organized into groups of different sizes depending on the scenario. For the single-processor scenario, each group included one student acting as the processor and one student acting as the timer. For the two-processor and four-processor scenarios, additional students were assigned as processors according to the task-partitioning strategy. Each group received printed flag grids and crayons. The student responsible for timing used their own phone as a stopwatch.

Each group was assigned a group number. After each scenario, the instructor recorded the completion time for each group on the whiteboard. This allowed the class to compare the time costs across groups and across different processor configurations. The timing results were mainly used to support discussion of sequential execution, parallel execution, speedup, scalability, and load balancing.

The instructor then used the follow-up questions and explanation slides to connect each

scenario to additional PDC concepts. Scenario 1 supported discussion of sequential processing and single-processor bottlenecks. Scenario 2 introduced task division and possible race conditions when processors are assigned overlapping work. Scenario 3 supported discussion of speedup, scalability, synchronization, and idle time caused by load imbalance. Scenario 4 introduced an alternative column-based partitioning strategy and was used to discuss boundary management, contention for shared resources, dependencies, distributed task partitioning, and pipelining.

The activity concluded with browser-based animations that reinforced the concepts demonstrated in the unplugged exercise. The animations helped connect the physical coloring task to computational models of parallel execution and provided a visual bridge between the classroom activity and PDC terminology.

Experience and Teaching Tips

- Print the flag grid at less than half of a letter-size page to reduce coloring time. This helps preserve more class time for comparing timing results and discussing the PDC concepts.
- Tell students before the pre-quiz that the questions are intended to measure prior knowledge and that incorrect answers are expected.
- Keep the coloring rules strict. If students color multiple squares at once, the model no longer represents processor-level work accurately.
- Use visible timing data. Write each scenario's completion time on the board so students can compare observed speedup with expected speedup.
- Ask students why the four-processor scenario may not produce exactly a fourfold improvement. This opens discussion of human variation and coordination overhead.
- When discussing contention, use classroom examples such as two processors needing the same marker or trying to work on the same grid boundary.
- Use Scenario 4 to show that partitioning strategy matters. The same number of processors can behave differently depending on how the data are divided.
- Use the animations after students complete the unplugged activity, not before. This preserves the discovery experience and then provides formal reinforcement.

Data and Analysis

A post-activity survey was administered after the Flag Maker activity to collect students' perceptions of the activity, group learning, conceptual understanding, engagement, and instructional support. The survey contained Likert-scale items and open-ended questions. A total of eight valid student responses were included in this analysis. Because of the small sample size, the results are interpreted descriptively rather than as inferential evidence.

For the Likert-scale items, responses were coded as follows for summary purposes: Neutral = 4, Somewhat agree = 5, Agree = 6, and Strongly agree = 7. No student selected a disagree

option for any of the Likert-scale items. Positive responses were defined as Somewhat agree, Agree, or Strongly agree. Table 9.7 summarizes the results by survey dimension.

Table 9.7: Summary of Flag Maker post-activity survey responses.

Survey Dimension	Items Included	Mean	Positive Responses
Peer and group learning	Explaining material, receiving explanations, group discussion, group contributions	5.19	96.9%
Engagement and preference	Having fun, preferring a class with the activity, focus, effort	5.19	87.5%
PDC and course learning	Confidence, understanding of parallel computing, understanding of loops, connection to programming assignment	4.91	81.3%
Student contribution	Valuable contribution to the group	5.25	87.5%
Instructional support	Instructor preparation, effort, enthusiasm, and availability of instructor or TA	5.72	93.8%

The strongest results were related to instructional support. Students rated the instructor's preparation and effort especially highly, with both items receiving a mean of 5.88 out of 6. The instructor or TA availability item also received a high mean score of 5.63. These results suggest that students perceived the activity as well organized and supported during implementation.

The group-learning results were also positive. All students agreed at least somewhat that having the material explained by group members improved their understanding, and all students agreed at least somewhat that group discussion contributed to their understanding of parallel computing. This supports the design choice of using an unplugged collaborative activity rather than presenting PDC concepts only through lecture.

Students also reported that the activity supported their understanding of PDC concepts. The item "The activity increased my understanding of parallel computing" had a mean of 5.13, with 87.5% positive responses. The item about understanding loops had a lower mean of 4.50, suggesting that students viewed the activity more strongly as a parallel-computing activity than as a direct reinforcement of loop concepts. This is reasonable because the CS1 version used the Flag Maker primarily as an unplugged PDC activity rather than as a plugged-in Java programming assignment.

Open-ended responses provide additional evidence that students connected the activity to PDC ideas. Several students described learning that multiple processors, represented by multiple people working together, can complete a task faster than a single processor. One student specifically mentioned pipelining and CPU performance, while another noted that changing how a system works can greatly increase efficiency. These comments suggest that the activity helped some students move beyond the surface-level coloring task and reason about system-level performance.

Overall, the Flag Maker post-activity survey suggests that students found the activity engaging, collaborative, and useful for understanding parallel computing. The results are exploratory because of the small number of responses, but they provide encouraging evidence that a short unplugged activity can help CS1 students reason about parallel execution, speedup, pipelining, and the role of coordination in multi-processor systems.

Table 9.8: Themes from open-ended Flag Maker activity responses.

Theme	Representative Evidence	Interpretation
Parallel execution and speedup	Students noted that multiple people working together completed the coloring task faster than one person working alone.	The activity made the benefit of parallel execution concrete.
Pipelining and system performance	One response identified pipelining as an interesting concept related to improving CPU performance.	Some students connected the activity to broader computer architecture ideas.
Efficiency through system design	One response noted that changing how a system works can greatly increase efficiency.	Students recognized that task organization affects performance.
Positive activity design	Several students wrote that no major improvement was needed or that the activity was well planned.	Students generally perceived the activity as clear and effective.

9.4 New Plugged-in Activities

No new plugged-in PDC programming activity was added to CS1 because the Java programming sequence at our institution is divided into two sequential courses: CSIT 111, Fundamentals of Programming I, and CSIT 112, Fundamentals of Programming II. CSIT 111 focuses on introductory programming foundations, including variables, expressions, conditionals, loops, arrays, classes, objects, and basic problem solving. At this stage, students are still developing basic programming fluency, so requiring them to implement PDC concepts with threads or other parallel programming tools would likely create too much cognitive load. Therefore, the Flag Maker activity was designed as an unplugged conceptual activity that introduces core PDC ideas without requiring parallel programming. Programming-based PDC activities are instead placed in our CS2, where students have stronger Java preparation and can work with plugged-in activities such as multithreaded image processing.

9.5 Course-wide Pre and Post Course Surveys

To evaluate the effectiveness of integrating parallel and distributed computing (PDC) concepts throughout the CS1 course, we administered matched pre- and post-course surveys at the beginning and end of the semester.

The survey measured students' self-reported understanding of fundamental programming concepts, PDC concepts, and attitudes toward computing. Students rated their level of understanding of the following programming concepts:

1. Data/Variable Types
2. Arithmetic Expressions and Operators
3. Branching
4. Loops
5. Calling Functions/Methods
6. Arrays

7. Writing Static Functions
8. Writing Classes
9. Writing Methods
10. Creating Objects

To evaluate students' understanding of PDC, they also rated their familiarity with the following concepts:

1. Parallel Processing
2. Distributed Computing
3. Event Handling

Finally, students responded to the following attitude statements:

A1 In general, my interest in computing is high.

A2 My interest in learning about parallel and distributed computing is high.

A3 In general, I believe learning about parallel and distributed computing is important.

A4 I believe I have a good understanding of computing.

A5 I believe I have a good understanding of parallel and distributed computing.

All survey items were measured using a seven-point Likert scale. Programming and PDC concept questions ranged from 1 = No Experience at All to 7 = Extremely Experienced, while attitude questions ranged from 1 = Strongly Disagree to 7 = Strongly Agree.

As shown in Table 9.9, the comparison between the no-intervention semester and the FlagMaker intervention semester suggests that the FlagMaker activity did not negatively affect students' learning of basic programming skills. Students in both semesters reported gains from pre-survey to post-survey across all basic programming categories. The no-intervention semester showed slightly larger gains for several introductory topics, such as data types, arithmetic expressions, branching, and writing static functions. However, the intervention semester had comparable post-survey means, and in some cases slightly higher post-survey means, such as loops, writing methods, and writing objects. This suggests that adding the unplugged FlagMaker activity did not reduce students' perceived progress in the core CS1 programming content.

Table 9.9: CS1 survey results for basic programming skills.

	No intervention			Intervention		
Basic Programming Skill	R_{base_pre}	R_{base_post}	$Mdiff_{base}$	R_{int_pre}	R_{int_post}	$Mdiff_{int}$
Data types	3.12	4.50	1.38	3.33	4.33	1.00
Arithmetic expressions and operators	2.75	4.62	1.88	3.00	4.44	1.44
Branching	2.75	4.50	1.75	3.00	4.44	1.44
Loops	3.00	4.75	1.75	3.22	4.78	1.56
Calling functions/methods	3.38	3.88	0.50	3.56	4.00	0.44
Arrays	2.50	3.62	1.12	2.78	3.62	0.85
Writing static functions	2.50	4.12	1.62	2.78	4.11	1.33
Writing classes	3.50	4.50	1.00	3.67	4.44	0.78
Writing methods	2.88	4.38	1.50	3.11	4.44	1.33
Writing objects	3.12	4.88	1.75	3.33	4.89	1.56

For PDC concepts, both semesters showed gains from pre-survey to post-survey, as shown in Table 9.10. The intervention semester had higher post-survey means than the no-intervention semester for all three PDC-related concepts: parallel processing, distributed computing, and event handling. Although the mean gains were larger in the no-intervention semester, students in the intervention semester entered with higher pre-survey ratings. The higher post-survey means in the Flag Maker semester suggest that students finished the course with slightly stronger self-reported understanding of these PDC-related topics.

Table 9.10: CS1 survey results for PDC concepts.

	No intervention			Intervention		
PDC Concept	R_{base_pre}	R_{base_post}	$Mdiff_{base}$	R_{int_pre}	R_{int_post}	$Mdiff_{int}$
Parallel processing	1.29	2.62	1.34	1.75	2.78	1.03
Distributed computing	1.14	2.75	1.61	1.62	3.00	1.38
Event handling	1.57	3.38	1.80	2.00	3.44	1.44

The attitude and belief results were mixed, as summarized in Table 9.11. Students in both semesters reported high interest in computing and high belief in the importance of learning PDC. Because these ratings were already relatively high at the beginning of the semester, there was limited room for large improvement. The intervention semester showed similar or slightly lower gains in general interest and perceived importance, but students' self-reported understanding of PDC was slightly higher at the end of the intervention semester than in the no-intervention semester. Overall, the comparison suggests that the FlagMaker activity helped reinforce PDC awareness while preserving students' progress in basic programming skills.

Table 9.11: CS1 survey results for attitudes and beliefs.

	No intervention			Intervention		
Attitude / Belief	R_{base_pre}	R_{base_post}	$Mdiff_{base}$	R_{int_pre}	R_{int_post}	$Mdiff_{int}$
Interest in computing is high	5.00	5.88	0.88	5.00	5.67	0.67
Interest in learning about PDC is high	4.86	5.00	0.14	4.88	4.89	0.01
Learning about PDC is important	4.14	5.38	1.23	4.38	5.33	0.96
Good understanding of computing	5.14	4.62	-0.52	5.25	4.67	-0.58
Good understanding of PDC	2.14	3.38	1.23	2.50	3.56	1.06

In addition to the comparison between the no-intervention and intervention semesters, we conducted a more detailed analysis of the matched pre- and post-course survey responses from the intervention semester. Matched responses were analyzed using the non-parametric

Wilcoxon signed-rank test. Statistical significance was evaluated at $\alpha = 0.05$, and effect sizes were computed using Cliff's Delta (δ). Tables 9.12–9.14 report the pre- and post-course means, mean differences, Wilcoxon statistics (W), p-values, statistical significance, Cliff's Delta (δ), and effect size interpretations.

Table 9.12 shows that students reported improvement across all basic computing concepts during the intervention semester. The largest mean gains were observed for loops and creating objects, followed by arithmetic expressions and operators, branching, writing static functions, and writing methods. Branching and loops reached statistical significance at the $\alpha = .05$ level, suggesting that students made measurable progress in these foundational CS1 topics. Several other items did not reach statistical significance, likely due in part to the small matched sample size, but their positive mean differences and medium to large effect sizes still suggest meaningful improvement in students' perceived programming knowledge.

Table 9.12: Matched pre- and post-course survey results for basic computing concepts in CS1.

Item	**Pre**	**Post**	**Diff**	W	p**-value**	**Sig.**	**Cliff's** δ	**Interp.**
Data/Variable Types	3.33	4.33	1.00	3.5	0.188	No	-0.28	small
Arithmetic Expressions and Operators	3.00	4.44	1.44	4.5	0.062	No	-0.49	large
Branching	3.00	4.44	1.44	2.5	0.039	Yes	-0.43	medium
Loops	3.22	4.78	1.56	0.0	0.016	Yes	-0.49	large
Calling Functions/Methods	3.56	4.00	0.44	1.0	0.500	No	-0.16	small
Arrays	2.50	3.62	1.12	7.5	0.188	No	-0.39	medium
Writing Static Functions	2.78	4.11	1.33	4.0	0.055	No	-0.40	medium
Writing Classes	3.67	4.44	0.78	1.5	0.188	No	-0.25	small
Writing Methods	3.11	4.44	1.33	1.5	0.094	No	-0.46	medium
Creating Objects	3.33	4.89	1.56	0.0	0.062	No	-0.44	medium

Table 9.13 summarizes the matched pre/post results for PDC-related concepts. Students reported higher post-course understanding for all three PDC concepts: parallel processing, distributed computing, and event handling. Although none of these gains reached statistical significance, the effect sizes were medium for parallel processing and event handling and large for distributed computing. This pattern suggests that the Flag Maker activity and related course discussion helped increase students' awareness of PDC concepts, even though the short unplugged intervention and small sample size limited the statistical strength of the results.

Table 9.13: Matched pre- and post-course survey results for PDC concepts in CS1.

Item	**Pre**	**Post**	**Diff**	W	p**-value**	**Sig.**	**Cliff's** δ	**Interp.**
Parallel Processing	1.75	2.62	0.88	2.0	0.250	No	-0.42	medium
Distributed Computing	1.62	2.88	1.25	0.0	0.125	No	-0.50	large
Event Handling	2.00	3.12	1.12	7.5	0.344	No	-0.45	medium

Table 9.14 presents the results for students' attitudes and perceptions. Students entered the course with relatively high interest in computing and high interest in learning about PDC, leaving limited room for additional growth in these areas. Interest in computing increased slightly, while interest in learning PDC remained unchanged. Students' perceived importance of learning PDC increased, and their self-reported understanding of PDC increased from

2.50 to 3.50, with a medium effect size. The small decrease in self-reported understanding of computing may reflect a recalibration effect, where students became more aware of the complexity of computing after completing more course material.

Table 9.14: Matched pre- and post-course survey results for attitudes and perceptions in CS1.

Item	Pre	Post	Diff	W	p-value	Sig.	Cliff's δ	Interp.
Interest in Computing	5.00	5.50	0.50	5.0	0.344	No	-0.25	small
Interest in Learning PDC	4.88	4.88	0.00	7.5	1.000	No	0.06	negligible
Importance of Learning PDC	4.38	5.12	0.75	2.0	0.250	No	-0.25	small
Understanding of Computing	5.25	4.75	-0.50	1.5	0.375	No	0.31	small
Understanding of PDC	2.50	3.50	1.00	2.5	0.078	No	-0.41	medium

Note. Responses were coded on a seven-point Likert scale. Diff represents the post-course mean minus the pre-course mean. Wilcoxon signed-rank tests were two-sided, with statistical significance evaluated at $\alpha = .05$. Cliff's δ was calculated as $\delta(\text{pre}, \text{post})$; therefore, negative values indicate that post-course scores tended to be higher than pre-course scores. The matched sample size was $n = 9$ for most computing-concept items and $n = 8$ for items containing one missing matched response.

Because only eight to nine matched responses were available for each survey item, the statistical analyses had limited power. Therefore, the results should be interpreted as exploratory. Although only a small number of items reached statistical significance, the positive mean differences and moderate to large effect sizes across several programming and PDC concepts suggest that students' perceived understanding generally improved during the intervention semester.

9.6 Other Activities and Ideas

No additional PDC activities were added to CS1 because the course schedule was already very tight. CS1 is the first course in the Java programming sequence, and most class time is needed for foundational programming topics. Adding more PDC activities would have required removing or reducing essential CS1 content.

For this reason, the Flag Maker activity was designed as a short unplugged activity that could introduce core PDC ideas without requiring major changes to the course structure. More programming-based PDC activities are better suited for CS2, where students have stronger Java preparation and more experience with object-oriented programming.

9.7 Conclusion

The CS1 Flag Maker activity demonstrates that PDC concepts can be introduced in a CS1 course through a short, accessible, unplugged intervention. The revised course preserves the standard Java programming sequence while adding explicit exposure to data parallelism and multi-core thinking. Students experience the limitations of sequential execution, the performance benefits of parallel task partitioning, and the challenges that arise when multiple

processors must coordinate their work. Survey results show positive gains in students' self-reported PDC understanding, supporting the value of integrating PDC concepts early in the programming curriculum.

The delays and low participation stemming from the IRB process argue for submitting protocols well before the intervention semester and for building alternative, IRB-permissible incentives into the initial application, such as raffles, or making the activity itself a regular in-class exercise so that research participation requires only consent to use data already being generated, rather than extra effort from students. Finally, consistent adoption across sections taught by both full-time faculty and part-time lecturers is best achieved by lowering the cost of participation: a plug-and-play instructor packet with slides, survey link, lesson plan, code, and the assessment instruments; a brief virtual training session at the start of the semester; and endorsement and acknowledgment from the course coordinator and department administration during department meetings, which converts individual instructor buy-in into a shared course-level commitment.

With these refinements, the Flag Maker activity can scale beyond a single instructor's classroom to a durable, department-wide component of the CS1 curriculum, offering a low-cost and transferable model for other institutions seeking to bring PDC concepts into introductory courses.

Bibliography

[1] John Lewis and William Loftus. Revel for Java Software Solutions: Foundations of Program Design. Pearson, 10th edition, 2025. REVEL Access Code Card.

Appendix

Github Link to Instructional Material

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter9-MSU

Detailed Pre and Post Syllabus

The Parallel and Distributed Computing (PDC) intervention was implemented in CSIT 111, Fundamentals of Programming I, the CS1 course at Montclair State University. CSIT 111 is a Java-based introductory programming course covering fundamental programming concepts such as data and expressions, using and writing classes and objects, conditionals and loops, object-oriented design, arrays, and inheritance. The Spring 2025 semester served as the baseline semester before the PDC intervention was introduced. The Fall 2025 semester incorporated the Flag Maker unplugged activity as the PDC intervention.

The pre-intervention syllabus followed the standard CS1 structure and focused on introductory programming and object-oriented programming topics without an explicit PDC activity. The post-intervention syllabus preserved the existing CS1 course structure while integrating the Flag Maker activity into the later part of the semester. The Flag Maker activity was implemented on December 3, 2025, after students had developed basic programming knowledge needed to discuss computation, task division, and program execution. Through this activity, students explored sequential execution, parallel execution, data parallelism, speedup, scalability, synchronization, load balancing, race conditions, shared resources, and task partitioning. The pre- and post-course experience surveys were collected during the baseline and Flag Maker semesters to compare student experience, attitudes, and PDC-related learning outcomes. In this way, the PDC intervention was added to an otherwise standard CS1 sequence without displacing existing core programming content.

Organization of Material

The `chapter9-MSU` directory is organized as follows:

```
chapter9-MSU/
  COURSE_CALENDAR.md
  Detailed_Pre_and_Post_Syllabus/
  Evaluation_Data_and_Analysis/
  Plugged_Activities/
  Unplugged_Activities/
  README.md
```

COURSE_CALENDAR.md

Purpose: This file (available at `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/blob/main/CS1/chapter9-MSU/COURSE_CALENDAR.md`) provides the Fall 2025 course calendar for CSIT 111 at Montclair State University.

Contents: A week-by-week schedule of course modules, class meetings, labs, homework assignments, examinations, breaks, and the Flag Maker PDC activity.

Usage: Primary reference for understanding where the PDC intervention was placed within the overall CS1 course sequence.

Detailed_Pre_and_Post_Syllabus

Purpose: This directory (available at `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter9-MSU/Detailed_Pre_and_Post_Syllabus`) contains the pre- and post-intervention syllabus materials.

Contents: The Spring 2025 pre-PDC syllabus and the Fall 2025 post-PDC syllabus for CSIT 111.

Usage: Reference materials for comparing the baseline CS1 course structure with the semester in which the Flag Maker PDC activity was implemented.

Evaluation_Data_and_Analysis

Purpose: This directory (available at `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter9-MSU/Evaluation_Data_and_Analysis`) contains the anonymized and aggregate evaluation materials used to assess the intervention.

Contents: Aggregate analysis outputs comparing the Spring 2025 baseline semester and the Fall 2025 Flag Maker semester, including student programming experience, attitudes and beliefs, and PDC-related survey results.

Usage: Source for evaluation summaries, comparison tables, and analysis of student learning and experience before and after the PDC intervention.

Plugged_Activities

Purpose: This directory (available at `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter9-MSU/Plugged_Activities`) documents that no plugged activity was implemented as part of this CS1 course package.

Contents: A short README noting that the CS1 intervention used the Flag Maker unplugged activity rather than a plugged programming activity.

Usage: Clarifies the activity structure for readers and directs them to the unplugged Flag Maker materials.

Unplugged_Activities

Purpose: This directory (available at `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter9-MSU/Unplugged_Activities`) contains the unplugged PDC activity used in the CS1 course.

Contents: The Flag Maker activity materials, including the activity README and the instructor slide deck `CS1_MSU_FlagMaker_slides.pptx`.

Usage: Classroom materials for implementing the Flag Maker activity and introducing PDC concepts through a guided, hands-on, non-programming exercise.

README.md

Purpose: Quick reference documentation for the MSU CS1 course package.

Contents: Overview of the course package, instructor information, activity summary, repository organization, licensing information, and citation placeholder.

Usage: Entry point for users to understand the module's organization and purpose.

Survey and Other Anonymized Data and Analysis

On the GitHub link (`https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter9-MSU`), the directory "Evaluation_Data_and_Analysis" contains anonymized aggregate analysis files for evaluating the PDC intervention. The Spring 2025 semester served as the baseline semester, and the Fall 2025 semester included the Flag Maker activity. The analysis materials include comparison outputs for student programming experience, attitudes and beliefs, and PDC-related learning outcomes. Raw student-level survey files and open-ended responses are not included in the public repository in order to protect student privacy.